

# Operating System Introduction

## Questions and answers

### Question 1

What are the two main functions of an operating system?

### Answer 1

An operating system has two main functions.

- It provides users and programs with an **extended machine**. This means it hides messy hardware details and gives cleaner abstractions, such as files, processes, virtual memory, and sockets.
- It acts as a **resource manager**. It controls and allocates CPU time, memory, disks, printers, network devices, and other system resources.

These two roles are related, but they are not identical. One is about abstraction; the other is about controlled sharing.

### Question 2

In Section 1.4, nine different types of operating systems are described. Give a list of applications for each of these systems, one per operating system type.

### Answer 2

Examples include the following.

| Operating system type              | Example application                                      |
|------------------------------------|----------------------------------------------------------|
| Mainframe operating system         | Claims processing in an insurance company.               |
| Server operating system            | Speech-to-text conversion service for a voice assistant. |
| Multiprocessor operating system    | Video editing, rendering, and encoding.                  |
| Personal computer operating system | Word processing or spreadsheet work.                     |
| Handheld computer operating system | Context-aware recommendation or navigation app.          |
| Embedded operating system          | Controlling a DVD recorder or smart appliance.           |
| Sensor-node operating system       | Monitoring temperature in a wilderness area.             |
| Real-time operating system         | Air traffic control or industrial control.               |
| Smart-card operating system        | Electronic payment or secure authentication.             |

### Question 3

What is the difference between timesharing and multiprogramming systems?

### Answer 3

A **multiprogramming system** keeps multiple programs in memory at the same time so that the CPU can switch to another program when one program blocks, usually for I/O. A **timesharing system** extends this idea to support multiple interactive users. Each user works at a terminal or session, and the system rapidly switches the CPU among users' processes to give the impression that everyone has a private machine.

All timesharing systems are multiprogramming systems, but not all multiprogramming systems are timesharing systems. For example, a single-user PC can run multiple programs at once without being a timesharing system for multiple users.

### Question 4

To use cache memory, main memory is divided into cache lines, typically 32 or 64 bytes long. An entire cache line is cached at once. What is the advantage of caching an entire line instead of a single byte or word at a time?

### Answer 4

Caching an entire line exploits **locality of reference**.

Programs often access memory locations near recently accessed locations. For example, after reading one instruction, the CPU will probably read the next instruction. After reading one array element, it will often read nearby array elements.

Therefore, if the cache brings in a full 32-byte or 64-byte line, future accesses are more likely to hit in the cache. Also, modern hardware can transfer a block of nearby memory into cache more efficiently than fetching the same bytes one word at a time.

### Question 5

On early computers, every byte of data read or written was handled by the CPU, that is, there was no DMA. What implications does this have for multiprogramming?

### Answer 5

The main benefit of multiprogramming is keeping the CPU busy while one process waits for I/O.

Without DMA, the CPU itself handles every byte transferred during I/O. If a process is doing I/O, the CPU is not idle; it is busy copying data. In that case, multiprogramming does not improve CPU utilization much, because there is no long CPU-free I/O wait to overlap with another program's execution.

This assumes the main I/O cost is copying data. If the I/O device is slow for another reason, such as waiting for bytes to arrive over a serial line, the CPU may still have time to run other processes.

### Question 6

Instructions related to accessing I/O devices are typically privileged instructions, that is, they can be executed in kernel mode but not in user mode. Give a reason why these instructions are privileged.

### Answer 6

I/O instructions are privileged because direct device access must be controlled by the operating system.

If user programs could directly issue I/O commands, they could bypass protection and policy. For example, one program might print without permission, read another user's disk data, corrupt a file system, or interfere with a network device.

By making I/O instructions privileged, the system forces user programs to request I/O through system calls. The operating system can then check permissions, enforce quotas, serialize access, and protect devices from misuse.

### Question 7

The family-of-computers idea was introduced in the 1960s with the IBM System/360 mainframes. Is this idea now dead as a doornail or does it live on?

### Answer 7

The idea is still alive.

A family of computers has compatible machines with different prices and performance levels. The same software can run across the family, even though the machines may differ in speed, cache size, number of cores, power use, and cost.

Modern CPU lines follow this idea. For example, processors in the same architecture family may include low-end, midrange, and high-end models. They differ in performance and price, but they are still architecturally compatible.

### Question 8

One reason GUIs were initially slow to be adopted was the cost of the hardware needed to support them. How much video RAM is needed to support a 25-line  $\times$  80-column character monochrome text screen? How much for a 1200  $\times$  900-pixel 24-bit color bitmap? What was the cost of this RAM at 1980 prices, \$5/KB? How much is it now?

### Answer 8

For a 25-line  $\times$  80-column character text screen:

$$25 \times 80 = 2000 \text{ characters}$$

Using 1 byte per character, this requires:

$$2000 \text{ bytes}$$

At \$5/KB, using 1 KB = 1024 bytes:

$$\frac{2000}{1024} \times 5 \approx \$9.77$$

So the text screen cost about:

$$\boxed{\$10}$$

For a 1200  $\times$  900-pixel 24-bit color bitmap:

$$1200 \times 900 \times 24 = 25,920,000 \text{ bits}$$

$$\frac{25,920,000}{8} = 3,240,000 \text{ bytes}$$

At \$5/KB:

$$\frac{3,240,000}{1024} \times 5 \approx \$15,820$$

So the bitmap cost about:

\$15,820

Using a rough modern RAM price of about \$15/GB, the text buffer costs far less than one cent, and the bitmap costs only about:

$$\frac{3,240,000}{1,073,741,824} \times 15 \approx \$0.045$$

That is about:

4.5 cents

The exact modern value changes with memory prices, but it is tiny compared with the 1980 cost.

### Question 9

There are several design goals in building an operating system, for example, resource utilization, timeliness, robustness, and so on. Give an example of two design goals that may contradict one another.

### Answer 9

One example is **fairness** versus **real-time responsiveness**.

Fairness tries to give each process a reasonable and balanced share of CPU time, memory, and I/O resources. Real-time responsiveness may require giving one process a disproportionate share of resources because it has a deadline.

For example, an air traffic control task may need CPU time immediately. A fair scheduler might want to divide CPU time equally, but a real-time scheduler must favor the time-critical task.

### Question 10

What is the difference between kernel and user mode? Explain how having two distinct modes aids in designing an operating system.

### Answer 10

In **kernel mode**, the CPU can execute all instructions and access all hardware features. The operating system kernel runs in this mode.

In **user mode**, the CPU is restricted. User programs cannot execute privileged instructions, directly access I/O devices, modify memory-management registers, or interfere with the kernel.

The two-mode design helps protect the system. User programs run with limited power. When they need an operating system service, they make a system call, which transfers

control to the kernel in a controlled way. This prevents ordinary programs from damaging the system or violating security rules.

### Question 11

A 255-GB disk has 65,536 cylinders with 255 sectors per track and 512 bytes per sector. How many platters and heads does this disk have? Assuming an average cylinder seek time of 11 ms, average rotational delay of 7 msec, and reading rate of 100 MB/sec, calculate the average time it will take to read 400 KB from one sector.

### Answer 11

Disk capacity is:

$$\text{capacity} = \text{cylinders} \times \text{heads} \times \text{sectors/track} \times \text{bytes/sector}$$

Solving for the number of heads:

$$\text{heads} = \frac{\text{capacity}}{65,536 \times 255 \times 512}$$

If the stated 255 GB is interpreted approximately as 255 GiB:

$$\text{heads} = \frac{255 \times 2^{30}}{65,536 \times 255 \times 512} = 32$$

Since each platter has two usable surfaces:

$$\text{platters} = \frac{32}{2} = 16$$

So the disk has:

32 heads and 16 platters

#### Capacity note

If the answer is given as 16 heads and 8 platters, that corresponds to about 137 GB decimal, not a 255 GB disk. For the capacity stated in this question, the consistent answer is 32 heads and 16 platters.

The average read time is:

$$\text{seek time} + \text{rotational delay} + \text{transfer time}$$

The transfer time for 400 KB at 100 MB/sec is approximately:

$$\frac{400 \text{ KB}}{100 \text{ MB/sec}} \approx 0.004 \text{ sec} = 4 \text{ ms}$$

Therefore:

$$11 \text{ ms} + 7 \text{ ms} + 4 \text{ ms} = 22 \text{ ms}$$

So the average time is:

22 ms

### Question 12

Which of the following instructions should be allowed only in kernel mode?

- (a) Disable all interrupts.
- (b) Read the time-of-day clock.
- (c) Set the time-of-day clock.
- (d) Change the memory map.

### Answer 12

The instructions that should be restricted to kernel mode are:

(a), (c), and (d)

Disabling interrupts, setting the clock, and changing the memory map can affect the whole system. If ordinary user programs could do these things, they could prevent scheduling, falsify system time, or access memory that does not belong to them.

Reading the time-of-day clock is normally safe for user programs because it only observes information and does not change system state.

### Question 13

Consider a system that has two CPUs, each CPU having two threads through hyperthreading. Suppose three programs,  $P_0$ ,  $P_1$ , and  $P_2$ , are started with run times of 5, 10, and 20 msec, respectively. How long will it take to complete the execution of these programs? Assume that all three programs are 100% CPU bound, do not block during execution, and do not change CPUs once assigned.

### Answer 13

The completion time depends on how the operating system assigns the programs to the physical CPUs.

For CPU-bound work, two hyperthreads on the same physical CPU share that CPU's execution resources. Thus, if two programs are assigned to the same physical CPU, their times effectively add.

- If  $P_0$  and  $P_1$  share one CPU, and  $P_2$  runs on the other CPU:

$$\max(5 + 10, 20) = \boxed{20 \text{ msec}}$$

- If  $P_0$  and  $P_2$  share one CPU, and  $P_1$  runs on the other CPU:

$$\max(5 + 20, 10) = \boxed{25 \text{ msec}}$$

- If  $P_1$  and  $P_2$  share one CPU, and  $P_0$  runs on the other CPU:

$$\max(10 + 20, 5) = \boxed{30 \text{ msec}}$$

If all three programs were assigned to the same physical CPU, the total would be:

$$5 + 10 + 20 = \boxed{35 \text{ msec}}$$

#### Question 14

A computer has a pipeline with four stages. Each stage takes the same time to do its work, namely, 1 nsec. How many instructions per second can this machine execute?

#### Answer 14

Once the pipeline is full, one instruction completes every 1 nsec.

$$1 \text{ instruction/nsec} = 1,000,000,000 \text{ instructions/sec}$$

So the machine can execute:

$$10^9 \text{ instructions/sec}$$

The number of pipeline stages affects latency for a single instruction, but the throughput is determined by how often a completed instruction leaves the pipeline.

#### Question 15

Consider a computer system that has cache memory, main memory (RAM), and disk, and an operating system that uses virtual memory. It takes 1 nsec to access a word from the cache, 10 nsec to access a word from RAM, and 10 ms to access a word from disk. If the cache hit rate is 95% and the main memory hit rate after a cache miss is 99%, what is the average time to access a word?

#### Answer 15

There are three cases.

- Cache hit: probability 0.95, cost 1 nsec.
- Cache miss but RAM hit: probability  $0.05 \times 0.99$ , cost 10 nsec.
- Cache miss and RAM miss, so disk is needed: probability  $0.05 \times 0.01$ , cost 10 ms = 10,000,000 nsec.

The average access time is:

$$(0.95)(1) + (0.05)(0.99)(10) + (0.05)(0.01)(10,000,000)$$

$$= 0.95 + 0.495 + 5000 = 5001.445 \text{ nsec}$$

So:

$$5001.445 \text{ nsec} \approx 5.001445 \mu\text{s}$$

#### Question 16

When a user program makes a system call to read or write a disk file, it provides an indication of which file it wants, a pointer to the data buffer, and the count. Control is then transferred to the operating system, which calls the appropriate driver. Suppose that the driver starts the disk and terminates until an interrupt occurs. In the case of reading from the disk, obviously the caller will have to be blocked because there are no data for

it. What about the case of writing to the disk? Need the caller be blocked awaiting completion of the disk transfer?

#### Answer 16

Not always, but the operating system must make sure the data remains valid until the disk write has actually happened.

If the caller is allowed to continue immediately and then overwrites the buffer, the disk might receive the changed data instead of the original data. Therefore, one simple solution is to block the caller until the write completes.

Another solution is for the kernel or driver to copy the data into a private kernel buffer before returning. Then the user process can continue safely because the disk write no longer depends on the user's buffer.

A third possibility is asynchronous I/O: the caller continues, but the system later notifies it when the buffer may be reused. This can work, but it is more complicated and easier to use incorrectly.

#### Question 17

What is a trap instruction? Explain its use in operating systems.

#### Answer 17

A **trap instruction** is a controlled instruction that transfers execution from user mode into kernel mode.

It is used to implement system calls. A user program cannot directly execute privileged operating system code, so it executes a trap instruction with information about the requested service. The CPU switches to kernel mode and jumps to a predefined kernel entry point.

The operating system then checks the request, performs the service if allowed, and returns control to the user program.

#### Question 18

Why is the process table needed in a timesharing system? Is it also needed in personal computer systems running UNIX or Windows with a single user?

#### Answer 18

The process table is needed because the operating system must remember the state of every process that is not currently running.

For each process, the table stores information such as registers, program counter, stack pointer, memory information, open files, scheduling state, and accounting data. When the CPU switches away from a process, this saved information is what allows the process to resume later.

It is also needed on single-user UNIX or Windows systems. Even when only one person is using the machine, many processes may be running: services, update checkers, network daemons, shells, editors, browsers, and background tasks. The operating system still needs a process table to manage them.



### Question 19

Is there any reason why you might want to mount a file system on a nonempty directory? If so, what is it?

### Answer 19

Yes, although mount points are normally empty.

When a file system is mounted on a directory, the old contents of that directory become hidden while the mount is active. This is usually undesirable.

However, an administrator might deliberately keep a few emergency files in the mount-point directory. If the mounted device fails or must be repaired, those files become visible again at the same path. This can provide fallback tools, configuration files, or recovery instructions when the normal mounted file system is unavailable.

### Question 20

For each of the following system calls, give a condition that causes it to fail: `fork`, `exec`, and `unlink`.

### Answer 20

Examples of failure conditions are:

- `fork` can fail if the system has no free process-table slots, or if there is not enough memory or swap space to create the new process.
- `exec` can fail if the named file does not exist, is not executable, or is not in a valid executable format.
- `unlink` can fail if the file does not exist, or if the caller lacks permission to remove it from its directory.

### Question 21

What type of multiplexing, time, space, or both, can be used for sharing the following resources: CPU, memory, disk, network card, printer, keyboard, and display?

### Answer 21

The resources can be classified as follows.

| Resource     | Typical multiplexing type        |
|--------------|----------------------------------|
| CPU          | Time multiplexing                |
| Memory       | Space multiplexing               |
| Disk         | Space multiplexing               |
| Network card | Time multiplexing                |
| Printer      | Time multiplexing                |
| Keyboard     | Time multiplexing                |
| Display      | Both time and space multiplexing |

The display is a good example of both: different windows may occupy different screen regions, and different programs may also update the display at different times.

### Question 22

Can the following call return any value in `count` other than `nbytes`? If so, why?

```
count = write(fd, buffer, nbytes);
```

### Answer 22

Yes. The return value can differ from `nbytes`.

If the call fails completely, it returns `-1`. For example, `fd` might be invalid, the process might not have permission to write, or the file system might report an error.

It may also return a smaller positive value if only part of the data was written. This can happen if the disk becomes full, a quota is reached, a signal interrupts the call, or the descriptor refers to something like a pipe, socket, or device that accepts only part of the data at that moment.

If the write completes normally and fully, then `count` is equal to `nbytes`.

### Question 23

A file whose file descriptor is `fd` contains the following sequence of bytes:

3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5

The following system calls are made:

```
lseek(fd, 3, SEEK_SET);  
read(fd, &buffer, 4);
```

where the `lseek` call makes a seek to byte 3 of the file. What does `buffer` contain after the read has completed?

### Answer 23

The file positions are numbered starting from 0.

| Position | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|----------|---|---|---|---|---|---|---|---|---|---|----|
| Byte     | 3 | 1 | 4 | 1 | 5 | 9 | 2 | 6 | 5 | 3 | 5  |

The call:

```
lseek(fd, 3, SEEK_SET);
```

moves the file position to byte 3. Reading 4 bytes then reads positions 3, 4, 5, and 6. Therefore, `buffer` contains:

1, 5, 9, 2

### Question 24

Suppose that a 10-MB file is stored on a disk on the same track, track 50, in consecutive sectors. The disk arm is currently situated over track number 100. How long will it take to retrieve this file from the disk? Assume that it takes about 1 ms to move the arm from one cylinder to the next and about 5 ms for the sector where the beginning of the file is stored to rotate under the head. Also, assume that reading occurs at a rate of 200 MB/sec.

#### Answer 24

The disk arm must move from track 100 to track 50:

$$100 - 50 = 50 \text{ tracks}$$

At 1 ms per track, the seek time is:

$$50 \times 1 \text{ ms} = 50 \text{ ms}$$

The rotational delay is given as:

$$5 \text{ ms}$$

The transfer time for a 10-MB file at 200 MB/sec is:

$$\frac{10}{200} \text{ sec} = 0.05 \text{ sec} = 50 \text{ ms}$$

Therefore, the total time is:

$$50 \text{ ms} + 5 \text{ ms} + 50 \text{ ms} = \boxed{105 \text{ ms}}$$

#### Question 25

What is the essential difference between a block special file and a character special file?

#### Answer 25

A **block special file** represents a device organized as numbered blocks. Each block can be read or written independently, and the system can seek to a specific block before doing I/O. Disks are the usual example.

A **character special file** represents a device that behaves more like a stream of characters or bytes. It is normally accessed sequentially, without arbitrary seeking to numbered blocks. Keyboards, terminals, and serial devices are common examples.

The essential difference is random block access versus stream-like sequential access.

#### Question 26

In the example given in Fig. 1-17, the library procedure is called **read** and the system call itself is called **read**. Is it essential that both of these have the same name? If not, which one is more important?

#### Answer 26

No. It is not essential that the library procedure and the kernel system call have the same name.

Inside the kernel, a system call is normally selected by a system-call number, not by a textual name. The library procedure places the appropriate system-call number and arguments where the kernel expects them, then executes a trap instruction.

The library procedure name is more important to the programmer because that is the name used in source code. The kernel interface may use numbers internally, but the programmer writes calls such as:

```
read(fd, buffer, nbytes);
```

### Question 27

Modern operating systems decouple a process address space from the machine's physical memory. List two advantages of this design.

### Answer 27

Two important advantages are:

- The same executable can be loaded at different physical memory locations on different runs. The program can use its own virtual addresses while the operating system maps those addresses to available physical frames.
- A program's virtual address space can be larger than physical RAM. With virtual memory, parts of the address space can be kept on disk and brought into RAM only when needed.

This design also helps with protection, because each process can have its own private address space.

### Question 28

To a programmer, a system call looks like any other call to a library procedure. Is it important that a programmer know which library procedures result in system calls? Under what circumstances and why?

### Answer 28

For program logic, it usually does not matter. A programmer can call the library procedure and rely on it to do the right thing.

For performance, it can matter a lot. A system call requires switching from user mode to kernel mode and later returning to user mode. That mode switch has overhead.

Also, after a system call, especially one that may block, the operating system may schedule another process to run. So if a task can be done entirely in user space, it may be faster than doing it through repeated system calls.

### Question 29

Figure 1-23 shows that a number of UNIX system calls have no Win32 API equivalents. For each of the calls listed as having no Win32 equivalent, what are the consequences for a programmer converting a UNIX program to run under Windows?

### Answer 29

Some UNIX calls do not map directly to Win32 calls, so the programmer must redesign those parts of the program.

- **link:** A UNIX program may depend on a file having multiple names in the file system. A direct conversion cannot assume that this works the same way. Programs that use **link** for lock-file creation also need a different locking method.
- **mount** and **umount:** A UNIX program may assume a single directory tree with file systems attached at mount points. On Windows, drive letters and different path conventions mean those assumptions may need to be changed.

- **chmod**: UNIX permission bits do not translate directly to Windows access control lists, so permission-handling code must be rewritten.
- **kill**: UNIX-style signal delivery does not translate directly. A Windows version needs a different way to stop, notify, or control another process.

### Question 30

A portable operating system is one that can be ported from one system architecture to another without any modification. Explain why it is infeasible to build an operating system that is completely portable. Describe two high-level layers that you would have in designing an operating system that is highly portable.

### Answer 30

A completely portable operating system is infeasible because hardware architectures differ in fundamental ways.

Different CPUs have different instruction sets, privilege mechanisms, interrupt handling, memory-management units, word sizes, and I/O buses. For example, code that manipulates page tables, handles traps, or initializes devices must know hardware-specific details.

A highly portable operating system can still be designed with two main layers:

- A **machine-dependent layer**, which contains the hardware-specific code: interrupt handling, low-level context switching, memory-management setup, and device-specific details.
- A **machine-independent layer**, which implements most of the operating system logic using the uniform interface provided by the machine-dependent layer.

To make the system portable, the machine-dependent layer should be kept as small and cleanly isolated as possible.

### Question 31

Explain how separation of policy and mechanism aids in building microkernel-based operating systems.

### Answer 31

Separation of policy and mechanism helps keep the microkernel small.

The kernel provides basic mechanisms, such as address spaces, interprocess communication, thread scheduling primitives, and low-level protection. These mechanisms are general and do not commit the system to one fixed high-level policy.

Policies, such as file-system behavior, scheduling choices, naming rules, or resource-allocation strategies, can then be implemented in user-level servers. This makes the system easier to modify because policy changes do not always require changing the kernel.

### Question 32

Virtual machines have become very popular for a variety of reasons. Nevertheless, they have some downsides. Name one.

### Answer 32

One downside is overhead.

A virtual machine adds a virtualization layer between the guest operating system and the real hardware. This can increase CPU overhead, memory usage, I/O overhead, and storage consumption compared with running directly on the hardware.

Modern virtualization is efficient, but it is not completely free.

### Question 33

Here are some questions for practicing unit conversions:

- (a) How long is a nanoyear in seconds?
- (b) Micrometers are often called microns. How long is a megam micron?
- (c) How many bytes are there in a 1-PB memory?
- (d) The mass of the earth is 6000 yottagrams. What is that in kilograms?

### Answer 33

- (a) A nanoyear is  $10^{-9}$  years:

$$10^{-9} \times 365 \times 24 \times 3600 = 0.031536 \text{ sec}$$

So:

$$0.031536 \text{ sec} = 31.536 \text{ msec}$$

- (b) A micron is  $10^{-6}$  meters. A megam micron is:

$$10^6 \times 10^{-6} \text{ m} = 1 \text{ meter}$$

- (c) If 1 PB is interpreted using binary memory units:

$$1 \text{ PB} = 2^{50} \text{ bytes}$$

$$1,125,899,906,842,624 \text{ bytes}$$

- (d) One yottagram is  $10^{24}$  grams. Therefore:

$$6000 \text{ Yg} = 6000 \times 10^{24} \text{ g} = 6 \times 10^{27} \text{ g}$$

Since  $1000 \text{ g} = 1 \text{ kg}$ :

$$6 \times 10^{27} \text{ g} = 6 \times 10^{24} \text{ kg}$$

### Question 34

Write a shell that is similar to Fig. 1-19 but contains enough code that it actually works so you can test it. You might also add some features such as redirection of input and output, pipes, and background jobs.

## Answer 34

The following small C shell supports ordinary commands, `cd`, `exit`, background jobs using `&`, input/output redirection using `<` and `>`, and one simple pipe.

```
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#define MAXLINE 1024
#define MAXARGS 64

static int tokenize(char *line, char *argv[])
{
    int argc = 0;
    char *tok = strtok(line, " \\t\\r\\n");

    while (tok != NULL && argc < MAXARGS - 1) {
        argv[argc++] = tok;
        tok = strtok(NULL, " \\t\\r\\n");
    }
    argv[argc] = NULL;
    return argc;
}

static void run_simple(char *argv[], int background)
{
    char *cmd[MAXARGS];
    int in = -1, out = -1, j = 0;

    for (int i = 0; argv[i] != NULL; i++) {
        if (strcmp(argv[i], "<") == 0 && argv[i + 1] != NULL) {
            in = open(argv[++i], O_RDONLY);
            if (in < 0) {
                perror("open input");
                return;
            }
        }
        else if (strcmp(argv[i], ">") == 0 && argv[i + 1] != NULL) {
            out = open(argv[++i], O_WRONLY | O_CREAT | O_TRUNC, 0666);
            if (out < 0) {
                perror("open output");
                if (in >= 0) close(in);
                return;
            }
        }
        else {
            cmd[j++] = argv[i];
        }
    }
    cmd[j] = NULL;

    if (cmd[0] == NULL) {
        if (in >= 0) close(in);
        if (out >= 0) close(out);
        return;
    }
}
```

```

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return;
    }
    if (pid == 0) {
        if (in >= 0) {
            dup2(in, STDIN_FILENO);
            close(in);
        }
        if (out >= 0) {
            dup2(out, STDOUT_FILENO);
            close(out);
        }
        execvp(cmd[0], cmd);
        perror(cmd[0]);
        _exit(127);
    }

    if (in >= 0) close(in);
    if (out >= 0) close(out);

    if (background)
        printf("[background pid %ld]\n", (long) pid);
    else
        waitpid(pid, NULL, 0);
}

static void run_pipe(char *argv[], int pipe_pos, int background)
{
    int fd[2];
    argv[pipe_pos] = NULL;

    if (argv[0] == NULL || argv[pipe_pos + 1] == NULL) {
        fprintf(stderr, "bad pipe syntax\n");
        return;
    }
    if (pipe(fd) < 0) {
        perror("pipe");
        return;
    }

    pid_t left = fork();
    if (left == 0) {
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]);
        close(fd[1]);
        execvp(argv[0], argv);
        perror(argv[0]);
        _exit(127);
    }

    pid_t right = fork();
    if (right == 0) {
        dup2(fd[0], STDIN_FILENO);

```



```

        close(fd[0]);
        close(fd[1]);
        execvp(argv[pipe_pos + 1], &argv[pipe_pos + 1]);
        perror(argv[pipe_pos + 1]);
        _exit(127);
    }

    close(fd[0]);
    close(fd[1]);

    if (!background) {
        waitpid(left, NULL, 0);
        waitpid(right, NULL, 0);
    }
}

int main(void)
{
    char line[MAXLINE];
    char *argv[MAXARGS];

    while (1) {
        while (waitpid(-1, NULL, WNOHANG) > 0)
            ;

        printf("mini-shell$ ");
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL)
            break;

        int argc = tokenize(line, argv);
        if (argc == 0)
            continue;

        if (strcmp(argv[0], "exit") == 0)
            break;

        if (strcmp(argv[0], "cd") == 0) {
            const char *dir = argc > 1 ? argv[1] : getenv("HOME");
            if (chdir(dir) < 0)
                perror("cd");
            continue;
        }

        int background = 0;
        if (argc > 0 && strcmp(argv[argc - 1], "&") == 0) {
            background = 1;
            argv[--argc] = NULL;
        }

        int pipe_pos = -1;
        for (int i = 0; i < argc; i++) {
            if (strcmp(argv[i], "|") == 0) {
                pipe_pos = i;
                break;
            }
        }
    }
}

```

```

    }
}

if (pipe_pos >= 0)
    run_pipe(argv, pipe_pos, background);
else
    run_simple(argv, background);
}

return 0;
}

```

This is intentionally small. It does not implement quoting, shell variables, wildcard expansion, job control, or multiple-stage pipelines. Those features are normally added by expanding the parser and the process-launching logic.

### Question 35

If you have a personal UNIX-like system available that you can safely crash and reboot, write a shell script that attempts to create an unlimited number of child processes and observe what happens. Before running the experiment, type `sync` to flush the file system buffers to disk. You can also do the experiment safely in a virtual machine. Do not try this on a shared system without permission.

### Answer 35

The experiment demonstrates that process creation is a finite resource. Eventually the system refuses new processes, usually with an error such as `Resource temporarily unavailable`, or the machine becomes so busy that it is hard to use.

#### Safety

Do not run an unbounded fork-bomb script on a shared machine or on a machine that contains work you care about. Use a disposable virtual machine and set a process limit first.

A safer version uses a process limit so the behavior can be observed without trying to exhaust the whole system:

```

#!/bin/sh

sync
ulimit -u 256 2>/dev/null

count=0
pids=""

while :
do
    (sleep 300) &
    status=$?

    if [ "$status" -ne 0 ]; then
        echo "fork failed after starting $count children"
        break
    fi
done

```

```

fi

pids="$pids $!"
count='expr "$count" + 1'

if [ 'expr "$count" % 25' -eq 0 ]; then
    echo "$count children started"
fi
done

kill $pids 2>/dev/null
wait 2>/dev/null

```

The important observation is that the operating system must protect itself by limiting the number of processes. When the per-user or system-wide process limit is reached, further attempts to create children fail.

### Question 36

Examine and try to interpret the contents of a UNIX-like or Windows directory with a tool like the UNIX `od` program. How you do this depends on what the operating system allows.

### Answer 36

A directory is not just a text file containing file names. It is an operating-system-specific data structure that maps names to file metadata.

On a UNIX-like file system, directory entries typically contain information such as:

- an inode number,
- a record length,
- a file-name length,
- sometimes a file type,
- the file name itself.

Modern systems often prevent ordinary programs from reading directory bytes directly, so a command such as:

```
od -c .
```

may fail with an error such as `Is a directory`. The usual portable interface is to use directory-reading calls such as `opendir` and `readdir`.

To inspect raw directory data, one safe approach is to create a small test file-system image, put a directory and a few files in it, unmount it, and then examine the image with a byte-dump tool:

```
od -Ax -tx1z imagefile
```

The output will contain binary metadata mixed with recognizable file names. The exact layout depends on the file system, such as FAT, ext, NTFS, or another format.